

PROJECT REPORT

Secure Orphanage Resource Management System

Database And Management System (UCS310)

Submitted by:

Icy Agarwal	1024030823
Harshit Taneja	1024030826
Riddhi Dhawan	1024030943

BE Second Year Batch: 2C61

Submitted to:

Ms. Banisha



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY,
PATIALA, PUNJAB
JAN-JUNE 2026

Submission Date: 1 May 2026

Contents

1.	INTRODUCTION	3
2.	PROBLEM STATEMENT.....	3
3.	OBJECTIVES OF THE PROJECT	4
4.	SCOPE OF THE PROJECT	4
5.	LIMITATIONS	4
6.	LITERATURE REVIEW.....	5
6.1	EXISTING SYSTEMS AND RELATED WORK	5
6.2	COMPARISON TABLE.....	5
6.3	RESEARCH GAP ADDRESSED	6
7.	PROPOSED SYSTEM DESCRIPTION	6
8.	DATABASE DESIGN	7
8.1	Relational Schema.....	7
9.	NORMALIZATION	8
10.	DATABASE IMPLEMENTATION.....	8
11.	TOOLS & TECHNOLOGIES USED.....	11
12.	RESULTS	11
13.	CONCLUSION.....	14
14.	REFERENCES	14

1. INTRODUCTION

The Secure Orphanage Resource Management System is a database-driven platform designed to manage child welfare records and resource distribution in orphanages. Orphanages handle highly sensitive and critical information, including personal details of children, medical history, educational progress, donations received, and allocation of essential resources such as food, clothing, and financial aid.

In many institutions, such information is still maintained through manual registers or loosely structured digital files. These approaches lead to redundancy, inconsistency, limited security, and difficulty in tracking resource utilization. A Database Management System (DBMS) provides a structured, centralized, and secure mechanism for handling such data efficiently.

This project emphasizes backend implementation using SQL and MySQL Stored Procedures, Functions, and Triggers. It incorporates normalization techniques to reduce redundancy, role-based access control to restrict unauthorized access, and audit logging with controlled database access to ensure confidentiality of sensitive child information. The system aims to ensure data integrity, transparency, and ethical management of orphanage operations.

2. PROBLEM STATEMENT

Orphanages require a structured system to manage welfare records and resources effectively. However, existing manual or file-based systems suffer from the following limitations:

- Data redundancy and inconsistency
- Lack of proper security for confidential child data
- Inefficient tracking of donations and resource allocation
- Absence of structured relationships between entities
- Limited accountability and transparency

There is a need for a secure, well-structured relational database system that ensures confidentiality, maintains referential integrity, and optimizes the allocation of available resources.

The project addresses this gap by developing a normalized database system that securely manages child records, donation tracking, and resource distribution using SQL and MySQL procedural automation.

3. OBJECTIVES OF THE PROJECT

The major objectives of the project are:

1. To design an Entity–Relationship (ER) model representing orphanage operations.
2. To convert the ER model into a relational schema with appropriate primary and foreign keys.
3. To apply normalization up to 3NF/BCNF to eliminate redundancy.
4. To implement the database using SQL (DDL and DML commands).
5. To develop advanced SQL queries including joins, subqueries, aggregate functions, and views.
6. To implement MySQL procedural components such as stored procedures, functions, and triggers.
7. To ensure data security using role-based access control, audit logging, and controlled database access.
8. To implement role-based access control for Admin, Staff, and Auditor roles.
9. To ensure transaction consistency using COMMIT, ROLLBACK, and SAVEPOINT commands.

4. SCOPE OF THE PROJECT

The scope of this project is limited to backend database implementation.

Functional Modules:

- Child Welfare Record Management
- Donation Management
- Resource Inventory Management
- Resource Allocation Tracking
- User Role & Access Control

Types of Users:

- **Admin:** Full database privileges
- **Staff:** Data entry and management access
- **Auditor:** Read-only access for transparency

The system focuses strictly on database – level implementation rather than front-end development.

5. LIMITATIONS

- The system is limited to backend database implementation and a basic web interface; no mobile application has been developed.
- Encryption of sensitive data is handled at the access control level and not at the column level (column-level encryption requires Oracle or enterprise-grade DBMS features).
- The system does not include biometric authentication or real-time notifications.

The frontend is a prototype demonstrating live database connectivity; it is not a production-grade application.

6. LITERATURE REVIEW

6.1 EXISTING SYSTEMS AND RELATED WORK

Child Welfare Information Gateway (USA) A federal system maintained by the U.S. Department of Health and Human Services for managing foster care and adoption records. It provides structured data management for child welfare but is limited to US jurisdiction and does not address resource allocation or donation tracking workflows relevant to orphanage management.

Hospital Management Systems (HMS) Systems such as OpenMRS and Bahmni address medical record management with normalization and role-based access control. Their data models inspired the design of our Children table with Medical Record attributes. However, these systems lack donation management and per-child resource allocation modules entirely.

NGO Resource Management Tools (Salesforce Nonprofit) Offers donor CRM features and donation tracking but is not tailored for orphanage-specific workflows. It does not support per-child resource allocation, audit logging through database triggers, or database-level procedural implementation using MySQL stored routines and triggers.

Academic Work — Database Design for Social Welfare Organizations Proposes ER models for NGOs covering donor-beneficiary relationships and basic normalization. However, it does not address MySQL procedural automation, triggers, role-based access control at the database level, or transaction management — all of which are core to our system.

6.2 COMPARISON TABLE

SYSTEM	CHILD RECORDS	DONATION TRACKING	RESOURCE ALLOCATION	RBAC	SQL/PL/SQL
Child welfare gateway	✓	✗	✗	✓	✗
Salesforce Nonprofit	Partial	✓	✗	✓	✗
OpenMRS	✓	✗	✗	✓	✗
Academic NGO model	✓	✓	✗	Partial	✗
Our system	✓	✓	✓	✓	✓

6.3 RESEARCH GAP ADDRESSED

- No existing system combines child welfare records, donation management, resource allocation, and MySQL procedural automation in a single backend database.
- Audit logging through database triggers is absent in all reviewed systems.
- Role-based access control enforced at the database level using GRANT statements is rarely implemented in academic NGO database models.
- Our system addresses all these gaps through an integrated, normalized MySQL implementation with automated triggers and transaction management.

7. PROPOSED SYSTEM DESCRIPTION

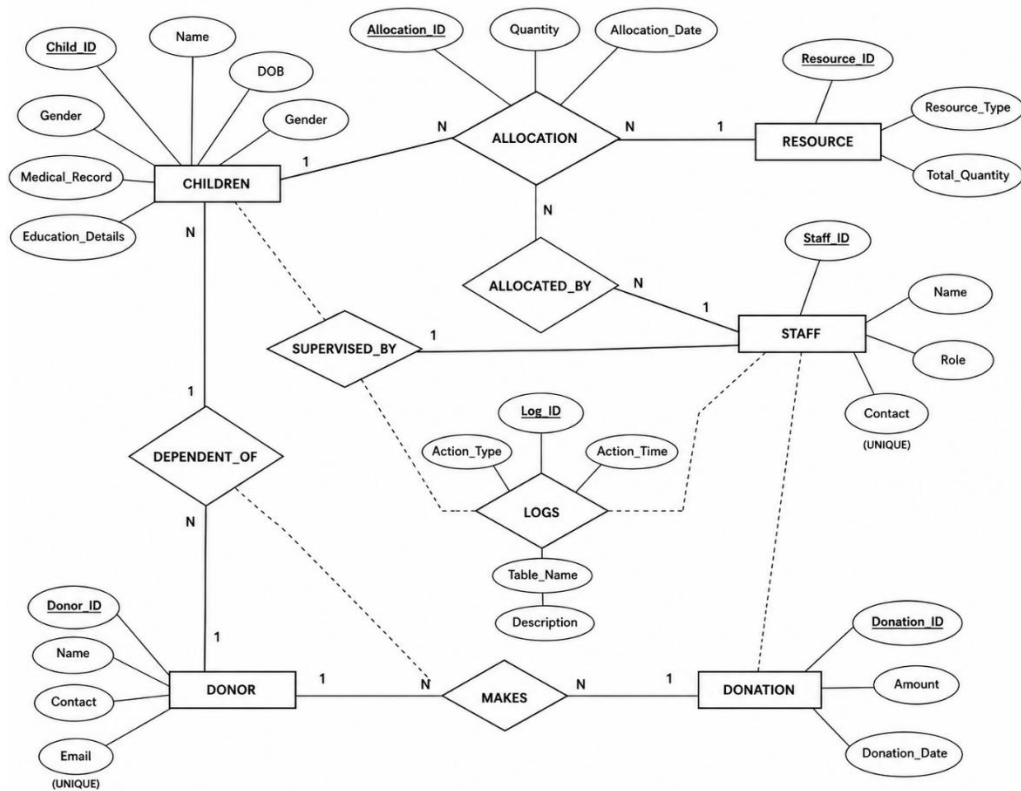
The proposed system is a secure relational database implemented using MySQL, Stored Procedures, Functions, and Triggers. It maintains structured records of children, staff, donations, resources, and allocations.

Key system features include:

- Secure storage of sensitive data using role-based access control and controlled database access.
- Role-based access restrictions
- Automated triggers for audit logging
- Efficient data retrieval using optimized SQL queries
- Transaction management to maintain consistency

The system improves operational efficiency by reducing redundancy and ensuring referential integrity through foreign key constraints. Automated stored procedures help manage repetitive operations such as allocation processing and reporting.

8. DATABASE DESIGN



8.1 Relational Schema

Children (Child_ID, Name, DOB, Gender, Medical_Record, education_Details)

Staff (Staff_ID, Name, Role, Contact)

Donor (Donor_ID, Name, Contact, Email)

Donation (Donation_ID, *Donor_ID*, Amount, Donation_Date)

Resource (Resource_ID, Resource_Type, Total_Quantity)

Allocation (Allocation_ID, *Child_ID*, *Resource_ID*, *Staff_ID*, Quantity, Allocation_Date)

Audit_Log (Log_ID, *Action_Type*, *Table_Name*, *Action_Time*, Description)

*Underline = Primary Key, Italics = Foreign Key

9. NORMALIZATION

First Normal Form (1NF)

All attributes in every table contain atomic, single-valued data. For example, Medical_Record stores one string per child, Gender uses an ENUM constraint ensuring a single controlled value, and no repeating groups exist anywhere in the schema.

Second Normal Form (2NF)

All non-key attributes are fully functionally dependent on the entire primary key. Since every table uses a single-column surrogate primary key (AUTO_INCREMENT), partial dependency is structurally impossible. For example, in the Allocation table, Quantity and Allocation_Date depend entirely on Allocation_ID, not on any subset of it.

Third Normal Form (3NF)

No transitive dependencies exist. Donor information (Name, Contact, Email) is stored in the Donor table and referenced in Donation via Donor_ID as a foreign key — donor details are never repeated inside Donation. Similarly, Staff details are not duplicated inside Allocation; only Staff_ID is stored as a reference.

Boyce-Codd Normal Form (BCNF)

Every determinant in every table is a candidate key. The UNIQUE constraints on Staff.Contact and Donor.Email make them candidate keys, and they determine only attributes directly belonging to their own entity. All tables satisfy BCNF conditions.

10. DATABASE IMPLEMENTATION

10.1 SQL Implementation

- **DDL commands**

```
CREATE TABLE Children (  
    Child_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Name VARCHAR(50) NOT NULL,  
    DOB DATE NOT NULL,  
    Gender ENUM('Male', 'Female', 'Other'),  
    Medical_Record VARCHAR(255),  
    Education_Details VARCHAR(255)  
);
```

- **DML commands (INSERT, UPDATE, DELETE)**

```
INSERT INTO Children (Name, DOB, Gender, Medical_Record, Education_Details) VALUES
('Aarav Sharma', '2012-03-14', 'Male', 'Mild Asthma', 'Class 6'),
('Ananya Verma', '2011-08-22', 'Female', 'No major medical history', 'Class 7'),
('Rohan Mehta', '2013-01-10', 'Male', 'Type 1 Diabetes', 'Class 5'),
('Priya Nair', '2010-11-05', 'Female', 'Iron deficiency anemia', 'Class 8'),
('Kabir Singh', '2012-06-18', 'Male', 'No major medical history', 'Class 6'),
('Meera Iyer', '2011-04-27', 'Female', 'Seasonal allergies', 'Class 7'),
('Arjun Patel', '2013-09-12', 'Male', 'No major medical history', 'Class 5'),
('Diya Kapoor', '2010-12-30', 'Female', 'Mild vision impairment', 'Class 8');
```

- **SELECT queries using joins**

```
174 • CREATE VIEW Allocation_Report AS
175 SELECT
176     a.Allocation_ID,
177     c.Name AS Child_Name,
178     r.Resource_Type,
179     a.Quantity,
180     s.Name AS Allocated_By,
181     a.Allocation_Date
182 FROM Allocation a
183 JOIN Children c ON a.Child_ID = c.Child_ID
184 JOIN Resource r ON a.Resource_ID = r.Resource_ID
185 JOIN Staff s ON a.Staff_ID = s.Staff_ID;
186
```

- **Subqueries**

```
SELECT Child_ID, SUM(Quantity) AS Total_Received
FROM Allocation
GROUP BY Child_ID
HAVING SUM(Quantity) >
(
    SELECT AVG(Quantity)
    FROM Allocation
);
```

- **Aggregate functions**

```
SELECT
    d.Name,
    SUM(do.Amount) AS Total_Donated
FROM Donor d
JOIN Donation do ON d.Donor_ID = do.Donor_ID
GROUP BY d.Name
ORDER BY Total_Donated DESC;
```

- **GROUP BY and HAVING clauses**

```
SELECT Child_ID, SUM(Quantity) AS Total_Received
FROM Allocation
GROUP BY Child_ID
HAVING SUM(Quantity) >
(
    SELECT AVG(Quantity)
    FROM Allocation
);
```

10.2 MySQL Procedural Components

```
CREATE TRIGGER trg_after_donation
AFTER INSERT ON Donation
FOR EACH ROW
BEGIN
    INSERT INTO Audit_Log(Action_Type, Table_Name, Description)
    VALUES(
        'INSERT',
        'Donation',
        CONCAT(
            'Donation of ₹',
            NEW.Amount,
            ' received from Donor_ID ',
            NEW.Donor_ID
        )
    );
);

DELIMITER //

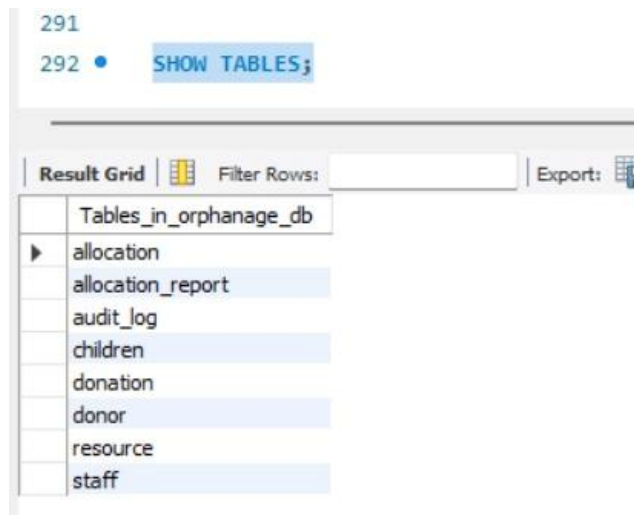
CREATE PROCEDURE AllocateResource(
    IN p_child_id INT,
    IN p_resource_id INT,
    IN p_quantity INT,
    IN p_staff_id INT
)
BEGIN
    INSERT INTO Allocation (
        Child_ID,
        Resource_ID,
        Quantity,
        Allocation_Date,
        Staff_ID
    )
    VALUES (
        p_child_id,
        p_resource_id,
        p_quantity,
        CURDATE(),
        p_staff_id
    );
END//
```

11.TOOLS & TECHNOLOGIES USED

- MySQL
- SQL
- MySQL Workbench
- MySQL Stored Procedures, Functions & Triggers

12. RESULTS

- SHOW TABLES



- DESC CHILDREN

The screenshot shows the MySQL Workbench interface. The SQL editor at the top contains the command `DESC Children;`. Below the editor, the 'Result Grid' tab is active, displaying the structure of the 'children' table. The table has the following fields: Child_ID (int, primary key, auto-increment), Name (varchar(50), multi-valued), DOB (date), Gender (enum('Male', 'Female', 'Other')), Medical_Record (varchar(255)), and Education_Details (varchar(255)).

Field	Type	Null	Key	Default	Extra
Child_ID	int	NO	PRI	NULL	auto_increment
Name	varchar(50)	NO	MUL	NULL	
DOB	date	NO		NULL	
Gender	enum('Male','Female','Other')	YES		NULL	
Medical_Record	varchar(255)	YES		NULL	
Education_Details	varchar(255)	YES		NULL	

- CHILDREN TABLE

304 • `SELECT * FROM CHILDREN;`

Child_ID	Name	DOB	Gender	Medical_Record	Education_Details
1	Aarav Sharma	2012-03-14	Male	Mild Asthma	Class 6
2	Ananya Verma	2011-08-22	Female	No major medical history	Class 7
3	Rohan Mehta	2013-01-10	Male	Type 1 Diabetes	Class 5
4	Priya Nair	2010-11-05	Female	Iron deficiency anemia	Class 8
5	Kabir Singh	2012-06-18	Male	No major medical history	Class 6
6	Meera Iyer	2011-04-27	Female	Seasonal allergies	Class 7
7	Arjun Patel	2013-09-12	Male	No major medical history	Class 5
8	Diya Kapoor	2010-12-30	Female	Mild vision impairment	Class 8
•	NULL	NULL	NULL	NULL	NULL

306 • `SELECT d.Name, SUM(do.Amount) AS Total_Donated`
 307 `FROM Donor d`
 308 `JOIN Donation do ON d.Donor_ID = do.Donor_ID`
 309 `GROUP BY d.Name`
 310 `ORDER BY Total_Donated DESC;`

Name	Total_Donated
Akshaya Patra Foundation	100000.00
Smile Foundation	75000.00
Goonj	40000.00
Seva Trust Patiala	25000.00
Anonymous Benefactor	15000.00

311
 312 • `SELECT * FROM Resource WHERE Total_Quantity < 100;`

Resource_ID	Resource_Type	Total_Quantity
2	School Uniform Sets	80
4	Medical Aid Kits	50
5	Winter Blankets	70
•	NULL	NULL

```

315 • SELECT Total_Quantity FROM Resource WHERE Resource_ID = 1;
316 • CALL AllocateResource(1, 1, 5, 1);
317 • SELECT * FROM Audit_Log;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	Total_Quantity			
▶	145			

```

315 • SELECT Total_Quantity FROM Resource WHERE Resource_ID = 1;
316 • CALL AllocateResource(1, 1, 5, 1);
317 • SELECT * FROM Audit_Log;

```

Result Grid		Filter Rows:	Edit:	Export/Import:	Wrap Cell Co
Log_ID	Action_Type	Table_Name	Action_Time	Description	
▶ 1	INSERT	Allocation	2026-05-01 21:13:43	Allocated 5 units of Resource_ID 1 to Child_ID 1	
2	INSERT	Allocation	2026-05-01 21:27:44	Allocated 5 units of Resource_ID 1 to Child_ID 1	
*	NULL	NULL	NULL	NULL	

```

322 • START TRANSACTION;
323 • INSERT INTO Donation(Donor_ID, Amount, Donation_Date) VALUES(2, 5000, CURDATE());
324 • ROLLBACK;
325 • SELECT * FROM Donation;

```

Result Grid		Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
Donation_ID	Donor_ID	Amount	Donation_Date		
▶ 1	1	75000.00	2026-03-15		
2	2	40000.00	2026-03-28		
3	3	100000.00	2026-04-05		
4	4	25000.00	2026-04-18		
5	5	15000.00	2026-04-25		
*	NULL	NULL	NULL		

13. CONCLUSION

The Secure Orphanage Resource Management System (SORMS) successfully demonstrates the practical application of core Database Management System concepts in a socially meaningful domain. By integrating SQL and MySQL procedural backend development with normalization up to BCNF, automated triggers, transaction management, and role-based access control, the project establishes a secure and efficient system for managing sensitive orphanage data.

The system achieves all stated objectives — a fully normalized 7-table relational schema, comprehensive DDL and DML implementation, advanced SQL queries using joins, subqueries, aggregate functions and views, three automated audit triggers, a stored procedure and function for procedural automation, four performance indexes, and a three-tier RBAC security model.

A key highlight of this project is the trigger-based audit logging system, which automatically records every allocation and donation event in real time without any manual intervention — ensuring complete transparency and accountability in orphanage operations.

Future extensions could include a web-based front-end interface, column-level encryption for Medical Record fields, multi-branch orphanage support, and integration with online payment gateways for donation collection.

14. REFERENCES

- [1] Ramakrishnan, R., & Gehrke, J. (2002). Database Management Systems (3rd ed.). McGraw-Hill.
- [2] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.
- [3] Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377–387.
- [4] Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.
- [5] MySQL 8.0 Reference Manual. Oracle Corporation.
<https://dev.mysql.com/doc/refman/8.0/en/>

[6] MySQL Stored Procedures and Functions. Oracle Corporation.

<https://dev.mysql.com/doc/refman/8.0/en/stored-routines.html>

[7] MySQL Triggers Documentation. Oracle Corporation.

<https://dev.mysql.com/doc/refman/8.0/en/triggers.html>

[8] MySQL Role-Based Access Control. Oracle Corporation

<https://dev.mysql.com/doc/refman/8.0/en/roles.html>.

[9] Child Welfare Information Gateway. U.S. Department of Health & Human Services.

<https://www.childwelfare.gov/>